

Nama : Pandu wiguna ruswandi (20240040258)

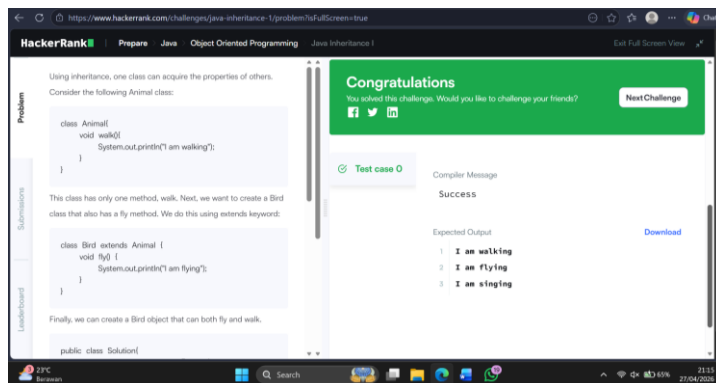
Kelas : TI24F

Matkul : OOP (praktikum)

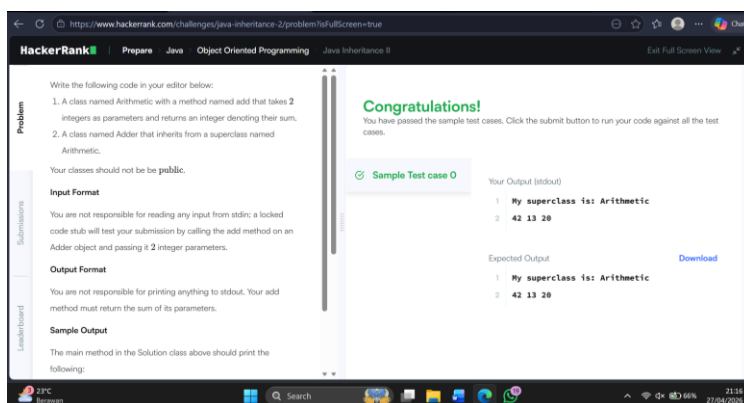
1. Buatlah akun di <https://www.hackerrank.com/>
2. Kerjakan soal latihan bagian <https://www.hackerrank.com/domains/java?filters%5Bsubdomains%5D%5B%5D=oop>
3. Screenshot masing - masing challenge lewat tab leader board
4. Kumpulkan semua screenshot dalam bentuk pdf
5. Upload file pdf ke edlink

Jawab

Java inheritance I



Java inheritance II



Java abstract class

The screenshot shows the HackerRank challenge page for 'Java Abstract Class'. The problem description explains that an abstract class is a class that cannot be instantiated and serves as a base for subclasses. It provides an example of an abstract class named 'Book' with a 'title' attribute and methods 'setTitle(String s)' and 'getTitle()'. A code editor on the left shows the abstract class definition. The right side of the page displays 'Congratulations!' and 'Sample Test case 0' and 'Sample Test case 1'. The test cases show input 'A tale of two cities' and the expected output 'The title is: A tale of two cities'.

Problem

A Java abstract class is a class that can't be instantiated. That means you cannot create new instances of an abstract class. It works as a base for subclasses. You should learn about Java Inheritance before attempting this challenge.

Following is an example of abstract class:

```
abstract class Book{
    String title;
    abstract void setTitle(String s);
    String getTitle();
    return title;
}
```

If you try to create an instance of this class like the following line you will get an error:

```
Book new_novel=new Book();
```

You have to create another class that extends the abstract class. Then

Sample Test case 0

Input (stdin)

```
A tale of two cities
```

Your Output (stdout)

```
The title is: A tale of two cities
```

Expected Output

```
The title is: A tale of two cities
```

Java interface

The screenshot shows the HackerRank challenge page for 'Java Interface'. The problem description explains that a Java interface can only contain method signatures and fields. It provides an example of an interface named 'AdvancedArithmetic' with a method signature 'divisorSum(int n)'. A code editor on the left shows the interface definition. The right side of the page displays 'Congratulations!' and 'Sample Test case 0', 'Sample Test case 1', and 'Sample Test case 2'. The test cases show input '6' and the expected output 'I implemented: AdvancedArithmetic' and '12'.

Problem

A Java interface can only contain method signatures and fields. The interface can be used to achieve polymorphism. In this problem, you will practice your knowledge on interfaces.

You are given an interface `AdvancedArithmetic` which contains a method signature `divisorSum(int n)`. You need to write a class called `MyCalculator` which implements the interface.

`divisorSum` function just takes an integer as input and return the sum of all its divisors. For example divisors of 6 are 1, 2, 3 and 6, so `divisorSum` should return 12. The value of `n` will be at most 1000.

Read the partially completed code in the editor and complete it. You just need to write the `MyCalculator` class only. Your class shouldn't be public.

Sample Input

```
6
```

Sample Output

```
I implemented: AdvancedArithmetic
12
```

Java method overriding

The screenshot shows the HackerRank challenge page for 'Java Method Overriding'. The problem description explains that when a subclass inherits from a superclass, it also inherits its methods. However, it can also override the superclass methods (as well as declare and implement new ones). It provides an example of a class named 'Sports' with methods 'getName()' and 'getNumberOfTeamMembers()'. A code editor on the left shows the 'Sports' class definition. The right side of the page displays 'Congratulations!' and 'Sample Test case 0'. The test cases show input 'Generic Sports' and the expected output 'Generic Sports', 'Each team has n players in Generic Sports', 'Soccer Class', and 'Each team has 11 players in Soccer Class'.

Problem

When a subclass inherits from a superclass, it also inherits its methods; however, it can also override the superclass methods (as well as declare and implement new ones). Consider the following `Sports` class:

```
class Sports{
    String getName(){
        return "Generic Sports";
    }
    void getNumberOfTeamMembers(){
        System.out.println("Each team has n players in " + get
    }
}
```

Next, we create a `Soccer` class that inherits from the `Sports` class. We can override the `getName` method and return a different, subclass-specific string:

```
class Soccer extends Sports{
    @Override
    String getName(){
        return "Soccer Class";
    }
}
```

Sample Test case 0

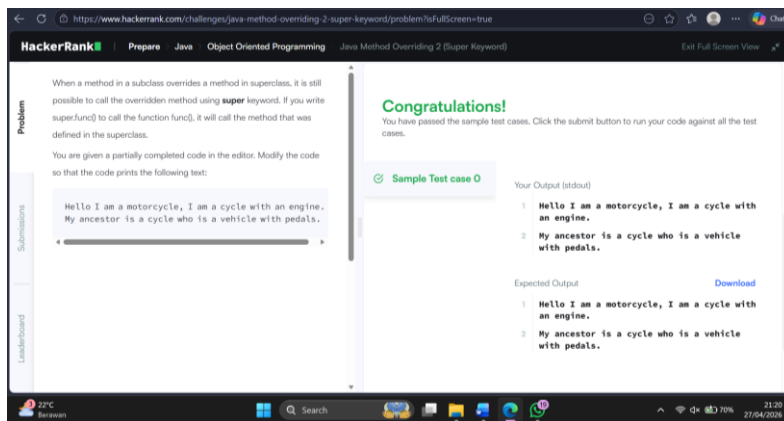
Your Output (stdout)

```
Generic Sports
Each team has n players in Generic Sports
Soccer Class
Each team has 11 players in Soccer Class
```

Expected Output

```
Generic Sports
Each team has n players in Generic Sports
Soccer Class
Each team has 11 players in Soccer Class
```

Java method overriding II



The screenshot shows the HackerRank interface for the 'Java Method Overriding 2 (Super Keyword)' challenge. The problem description explains that a method in a subclass can override a method in a superclass, and the `super` keyword can be used to call the overridden method. The provided code snippet shows a `Vehicle` class with a `say` method and a `Motorcycle` class that overrides it. The user's output matches the expected output, and the challenge is marked as 'Sample Test case 0'.

Problem

When a method in a subclass overrides a method in superclass, it is still possible to call the overridden method using `super` keyword. If you write `super.func()` to call the function `func()`, it will call the method that was defined in the superclass.

You are given a partially completed code in the editor. Modify the code so that the code prints the following text:

```
Hello I am a motorcycle, I am a cycle with an engine.
My ancestor is a cycle who is a vehicle with pedals.
```

Sample Test case 0

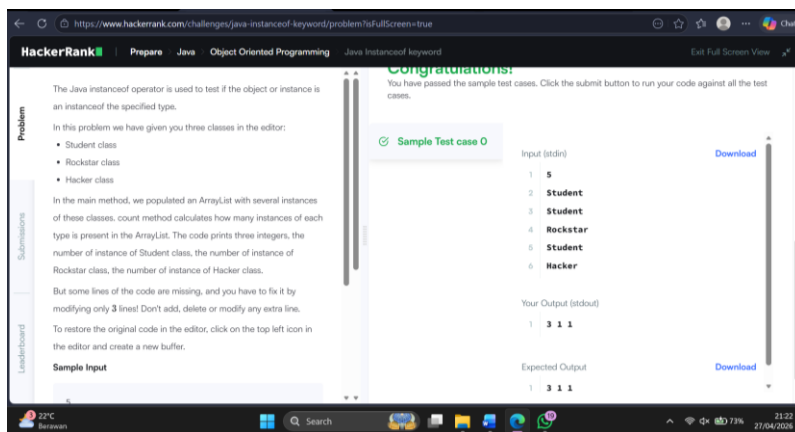
Your Output (stdout)

```
1 Hello I am a motorcycle, I am a cycle with an engine.
2 My ancestor is a cycle who is a vehicle with pedals.
```

Expected Output

```
1 Hello I am a motorcycle, I am a cycle with an engine.
2 My ancestor is a cycle who is a vehicle with pedals.
```

Java instanceof keyword



The screenshot shows the HackerRank interface for the 'Java instanceof keyword' challenge. The problem description explains that the `instanceof` operator is used to test if an object is an instance of a specified type. The provided code snippet shows a `main` method that iterates over an `ArrayList` of objects and prints the count of each type. The user's output matches the expected output, and the challenge is marked as 'Sample Test case 0'.

Problem

The Java `instanceof` operator is used to test if the object or instance is an instance of the specified type.

In this problem we have given you three classes in the editor:

- Student class
- Rockstar class
- Hacker class

In the main method, we populated an `ArrayList` with several instances of these classes. `count` method calculates how many instances of each type is present in the `ArrayList`. The code prints three integers, the number of instance of `Student` class, the number of instance of `Rockstar` class, the number of instance of `Hacker` class.

But some lines of the code are missing, and you have to fix it by modifying only 3 lines! Don't add, delete or modify any extra line. To restore the original code in the editor, click on the top left icon in the editor and create a new buffer.

Sample Input

```
5
Student
Student
Rockstar
Student
Hacker
```

Sample Test case 0

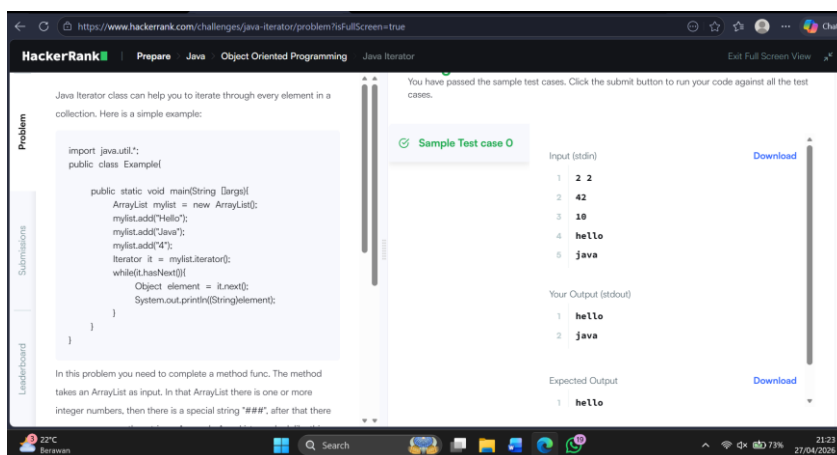
Your Output (stdout)

```
1 3 1 1
```

Expected Output

```
1 3 1 1
```

Java iterator



The screenshot shows the HackerRank interface for the 'Java iterator' challenge. The problem description explains that the `Iterator` class can help to iterate through every element in a collection. The provided code snippet shows a `main` method that iterates over an `ArrayList` of objects and prints the count of each type. The user's output matches the expected output, and the challenge is marked as 'Sample Test case 0'.

Problem

Java `Iterator` class can help you to iterate through every element in a collection. Here is a simple example:

```
import java.util.*;
public class Example {
    public static void main(String[] args) {
        ArrayList mylist = new ArrayList();
        mylist.add("Hello");
        mylist.add("Java");
        mylist.add("4");
        Iterator it = mylist.iterator();
        while(it.hasNext()) {
            Object element = it.next();
            System.out.println(String.valueOf(element));
        }
    }
}
```

In this problem you need to complete a method `func`. The method takes an `ArrayList` as input. In that `ArrayList` there is one or more integer numbers, then there is a special string `"###"`, after that there

Sample Test case 0

Your Output (stdout)

```
1 2 2
2 42
3 10
4 hello
5 java
```

Expected Output

```
1 hello
2 java
```